

Habit Tracker

Basit bir uygulama · **üretim kalitesinde** test & dağıtım altyapısı

115/100

Beklenen puan

+15

Bonus (tavan)

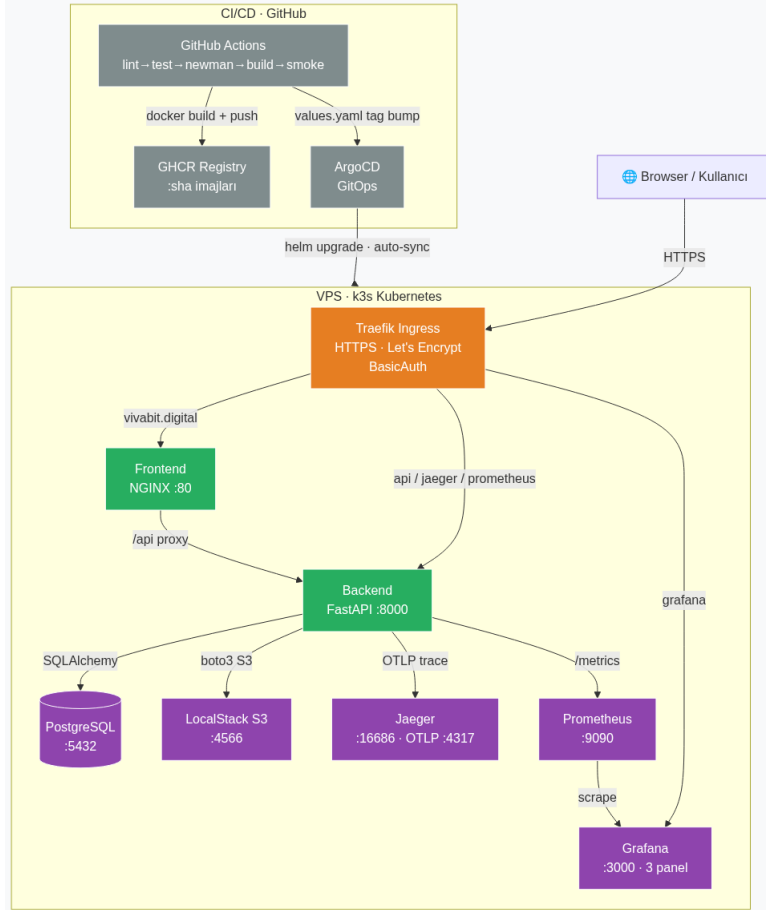
7

K8s servisi

58

Otomatik test

- **Problem:** Modern yazılımda asıl zorluk kod değil; testi, dağıtımı ve gözlemlenebilirliği **güvenilir + otomatik** hale getirmek
- **Seçilen uygulama:** Habit Tracker API — günlük alışkanlık takibi, streak, mood ve fotoğraf yükleme (S3)
 - 3 entity (User · Habit · HabitLog), 14 REST endpoint, JWT auth
- **Çözüm:** Uygulamayı sade tutup etrafına **üretim kalitesinde** bir test & CI/CD & monitoring iskeleti kurmak
 - Frontend (NGINX, static) + Backend (FastAPI, JSON) tamamen ayrı mikroservis
 - Her katman otomatik test → GitOps ile k3s cluster'a canlıya çıkıyor
- **Sonuç:** Şartnamenin tüm katmanları + 3 bonus (Helm · OpenTelemetry · ArgoCD)
- **Canlı:** <https://vivabit.digital> — gerçek bir VPS'te, HTTPS + Let's Encrypt



- **Traefik Ingress:** tek giriş, HTTPS (Let's Encrypt), host bazlı routing, BasicAuth
- **Frontend:** NGINX — static HTML/JS serve + /api reverse proxy (CORS yok)
- **Backend:** FastAPI — JSON REST, JWT, Prometheus /metrics
- **PostgreSQL:** User/Habit/HabitLog — kalıcı PVC
- **LocalStack S3:** alışkanlık fotoğrafları (boto3)
- **Jaeger:** OpenTelemetry OTLP ile distributed tracing
- **Prometheus + Grafana:** metrik toplama + 5 panel

Akış: Browser → Traefik → Frontend → /api → Backend → Postgres · S3 · Jaeger

Katman	Araç	Adet	Kapsam
Unit	pytest	8	auth, model, Factory Boy
Integration	pytest + TestClient	32	endpoint + auth + mood + foto
Testcontainers	gerçek PostgreSQL 16	3	kalıcılık doğrulama
E2E	Playwright	6	tarayıcıda kullanıcı akışı
API	Postman / Newman	7	CI'da canlı API
Performans	k6 (smoke + load)	2	p95 latency

- **Piramit mantığı:** çok sayıda hızlı unit, orta katmanda integration, az sayıda yavaş E2E
- **Coverage:** hedef %70 → ulaşılan **%86**; CI kapısı - -cov-fail-under=70 ile zorunlu (düşerse pipeline kırmızı)
- **Testcontainers:** gerçek PostgreSQL 16 container'ı → kalıcılık ve şema doğrulanır
- **Factory Boy + Faker:** her test izole, gerçekçi ve benzersiz veri üretir
- **58 otomatik test**, 6 farklı türde — hepsi CI'da her push'ta koşar

4

CI/CD Pipeline

GitHub Actions — 6 job, fail-fast zincir

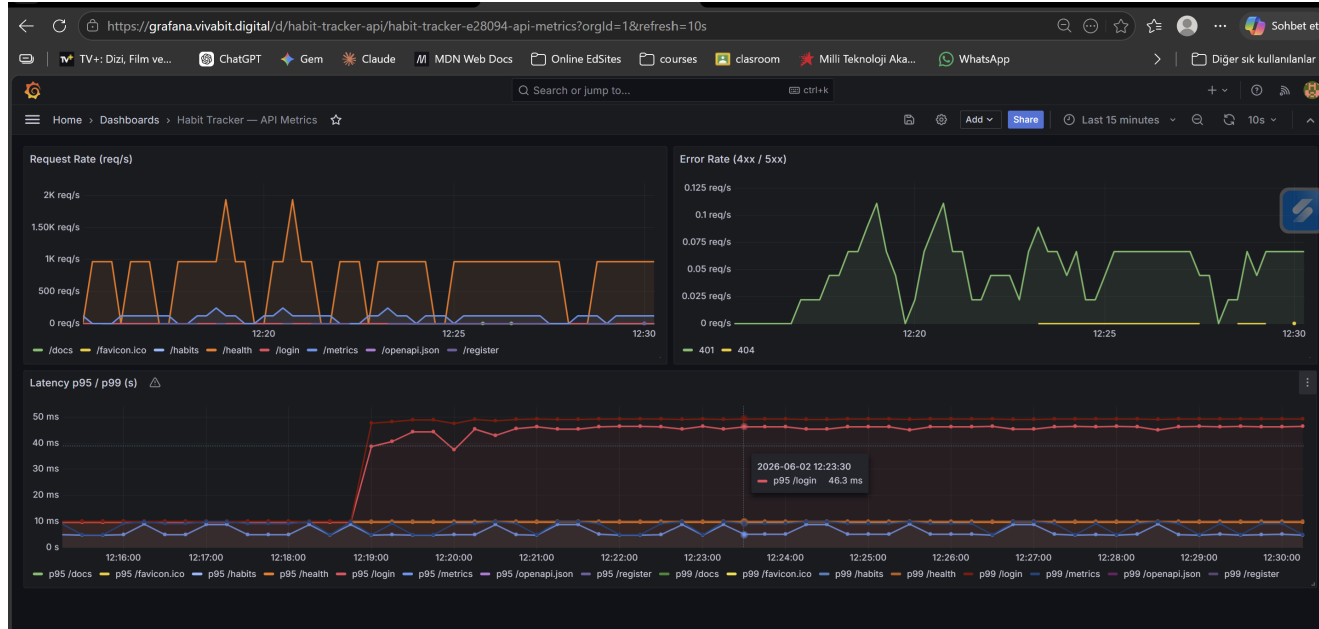


- **lint:** flake8 kod stili — en hızlı kapı, erken kırılır
- **test:** pytest + coverage (kapı %70) + Testcontainers (gerçek Postgres)
- **newman:** Postman koleksiyonu canlı API'ye (Postgres service) koşar
- **build:** backend + frontend imajları → GHCR (:sha, :latest)
- **deploy-smoke:** Kind cluster + helm template | kubectl apply + curl /health + k6
- **cd-bump:** Helm values.yaml image tag'ini yeni SHA'ya günceller → commit
- **Deploy stratejisi (GitOps):** push → CI → tag bump → **ArgoCD** farkı görüp k3s'e otomatik sync. Manuel kubectl yok; rollback = önceki revizyon / :sha imaj

5

Monitoring & Observability

Prometheus + Grafana + Jaeger (OpenTelemetry)



Grafana — 5 panel: request rate · error rate · p95/p99 latency · endpoint yük dağılımı · kullanıcı aktivitesi (canlı: grafana.vivabit.digital)

- **Prometheus:** `http_requests_total` (counter) + `request_duration` (histogram → p95/p99)
- **Jaeger (+5 bonus):** FastAPI + SQLAlchemy auto-instrument, OTLP ile distributed tracing
- **İncelik:** Service yerine her pod ayrı scrape edilir — yoksa 2 replikanın sayaçları karışıp `rate()` sahte spike üretir
- Tüm paneller Traefik BasicAuth arkasında: grafana · jaeger · prometheus . vivabit.digital

6

Sayılar

Projeyi tek bakışta özetleyen metrikler

58

otomatik test

%86

coverage

~285ms

p95 (yük testi)

7

K8s servisi

6

CI job

+15

bonus (Helm·OTel·ArgoCD)

- **p95 yorumu:** 285 ms, 500 ms eşiğinin çok altında — en yavaş uçlar /register ve /login (bcrypt hashing, güvenlik gereği optimize edilmez)
- **Şartname:** 100/100 zorunlu + 15/15 bonus (tavan) → beklenen 115/100

- **Servis ayrımı:** NGINX reverse proxy ile tek origin — CORS derdi yok
- **GitOps:** ArgoCD ile her push otomatik deploy; selfHeal manuel değişikliği geri alır
- **Gözlemlenebilirlik tuzağı:** Service'i scrape etmek 2 replikanın sayaçlarını karıştırıp sahte spike üretti → pod bazlı scrape ile çözüldü
- **Kalıcılık:** Postgres'e PVC eklenince restart'ta veri korunuyor
- **İleride:** KEDA ile event-driven autoscaling, Helm chart'ı registry'ye publish

- **1.** PR aç → GitHub Actions yeşil olmaya başlar
- **2.** PR merge → image build + push (GHCR)
- **3.** ArgoCD yeni versiyonu k3s'e otomatik sync eder
- **4.** Grafana'da deploy sonrası latency / error rate panelleri
- **5.** k6 ile kısa load testi → p95 latency
- **6.** Playwright ile 1–2 E2E senaryosu canlı koşar
- **Yedek:** canlı çökerse demo videosu

Teşekkürler

Sorular & canlı demo

Uygulama

vivabit.digital

API Docs

api.vivabit.digital/docs

Grafana

grafana.vivabit.digital

Tracing

jaeger.vivabit.digital